SAHAYAK Master Guide - Johan Jyoti Kuli

Updated operational handbook for the static and continuous sign pipelines, local backups, collection, testing, candidate promotion, GitHub model updates, and production verification.

Scope	Safe-by-default sequence
Windows local training project plus GitHub deployment repository.	Backup → collect → train candidate → test → promote → sync → Git commit/push → verify the deployed API.

1. Project identity and operating principle

Project: SAHAYAK - A Real-Time Sign Language Recognition System. **Developer:** Mr. Johan Jyoti Kuli. This guide is the operational index for the local training project, the separate GitHub deployment repository, and the two independent sign-recognition lanes.

Keep training data and development backups local. Promote an evaluated candidate only after tests and camera checks. GitHub should receive only the intended active model files and source/documentation changes.

2. Two model lanes - do not mix their files

Static signs: one-frame, two-hand landmark classification. Collection script: collect_data.py. Dataset: data\landmarks_2hand.csv. Candidate: model\sahayak_twohand_candidate.joblib plus labels_twohand_candidate.json. Active files: model\sahayak_twohand.joblib plus labels_twohand.json. Expected model input: 126 features.

Continuous signs: full movement sequences with temporal order. Collection script: collect_sequences.py. Database: continuous_gesture_db.db. Candidate: model\sahayak_sequence_candidate.joblib. Active file: model\sahayak_sequence.joblib. Expected model input: 1,320 temporal features. This uses sequence_db.py and sequence_features.py.

The older Code\gesture_db.db is a legacy database. Do not replace it or use it as the continuous sequence dataset. The static CSV and continuous SQLite database are separate stores.

3. Repository paths and deployment architecture

```
Local training project:
C:\Users\user\3D Objects\New folder\SAHAYAK

GitHub deployment repository:
https://github.com/Mrjohanjyotikuli/SAHAYAK-ISL

Separate local deployment clone (example):
C:\Users\user\3D Objects\New folder\SAHAYAK-ISL

Static deployment files:
SAHAYAK-Vercel-Deployment/backend/model/sahayak_twohand.joblib
SAHAYAK-Vercel-Deployment/backend/labels_twohand.json

Continuous deployment file:
SAHAYAK-Vercel-Deployment/backend/model/sahayak_sequence.joblib

Backend status:
https://sahayak-backend-krh1.onrender.com/api/status
Studio test page:
https://sahayak-eight-lac.vercel.app/studio.html
```

The training project and deployment clone must be different folders. sync_model_to_github.ps1 is the guarded copy helper: it copies only the promoted active model files for the selected ModelType. Do not copy the entire training project over the deployment backend.

4. Start in the existing Windows environment

```
Set-Location "C:\Users\user\3D Objects\New folder\SAHAYAK"
.\.venv\Scripts\Activate.ps1
python --version
Test-Path .\collect_data.py
Test-Path .\collect_sequences.py
Test-Path .\sync_model_to_github.ps1
```

If activation is blocked, use .\.venv\Scripts\python.exe in place of python. Avoid repeated reinstallations when the current environment already works.

5. Standard release cycle

Step A - back up. Copy the static CSV, current static model and label file, continuous SQLite database, and current sequence model to a timestamped local folder before collecting or promoting. Close any active sequence collection process before copying the SQLite file.

```
$stamp = Get-Date -Format "yyyyMMdd_HHmmss"
$backup = Join-Path $PWD "backups\release_$stamp"
New-Item -ItemType Directory -Path $backup -Force | Out-Null
foreach ($file in @(
  "data\landmarks_2hand.csv",
  "continuous_gesture_db.db",
  "model\sahayak_twohand.joblib",
  "model\labels_twohand.json",
  "model\sahayak_sequence.joblib"
)) { if (Test-Path $file) { Copy-Item $file $backup -Force } }
Write-Host "Backup: $backup"
```

Step B - collect. Run only the appropriate collector: static uses S to toggle capture and Q/Esc to exit; continuous uses C to start a full movement and C again to stop/save. Use consistent labels and varied real examples. Review actual per-label counts; do not enter assumed counts in the website or guide.

```
# Static collection and counts
python collect_data.py
@'
import csv, collections
with open('data/landmarks_2hand.csv', encoding='utf-8-sig', newline='') as f:
    rows = list(csv.DictReader(f))
print('TOTAL:', len(rows))
print('COUNTS:', dict(sorted(collections.Counter(r['label'] for r in rows).items())))
'@ | python -

# Continuous collection and counts
python collect_sequences.py
python -c "from sequence_db import summary; print(summary())"
```

Step C - train candidates. Training without --promote must leave the active model untouched.

```
# Static candidate
python train_model.py
python -m unittest test_sahayak.py

# Continuous candidate
python train_sequence_model.py --db
python -m unittest test_sequence.py test_sahayak.py
```

Step D - inspect. Confirm the intended new label is present, all previous labels remain, feature dimensions match, holdout metrics are reasonable, and real camera examples distinguish the new sign from old signs. Holdout results are estimates, not a guarantee.

Step E - promote the selected model only. Static: python train_model.py --promote. Continuous: python train_sequence_model.py --db --promote. Each promotes only its own lane and backs up its own active model first.

```
# Choose ONE of these after review:
python train_model.py --promote
# OR
python train_sequence_model.py --db --promote
```

Step F - local API and camera checks. Start python web_app.py, then query /api/status from another PowerShell window. Confirm model_ready/single_sign_labels for static and sequence_model_ready/sequence_sign_labels for continuous. Test the new and old signs locally before syncing.

6. GitHub model update - safe, explicit commands

```
$TrainingRoot = "C:\Users\user\3D Objects\New folder\SAHAYAK"
$RepoPath = "C:\Users\user\3D Objects\New folder\SAHAYAK-ISL"
if (-not (Test-Path (Join-Path $RepoPath ".git"))) {
  git clone https://github.com/Mrjohanjyotikuli/SAHAYAK-ISL.git $RepoPath
}
git -C $RepoPath status --short
# Stop if unexpected edits are present; save or resolve them first.
git -C $RepoPath pull --ff-only origin main
$backupBranch = "backup/pre-model-" + (Get-Date -Format "yyyyMMdd-HHmmss")
git -C $RepoPath branch $backupBranch main
git -C $RepoPath push origin $backupBranch
```

Static sign update (only after static candidate was promoted and checked):

```
Set-Location $TrainingRoot
.\sync_model_to_github.ps1 -RepoPath $RepoPath -ModelType Static
git -C $RepoPath status --short
git -C $RepoPath add `
  SAHAYAK-Vercel-Deployment/backend/model/sahayak_twohand.joblib `
  SAHAYAK-Vercel-Deployment/backend/labels_twohand.json
git -C $RepoPath diff --cached --name-only
git -C $RepoPath commit -m "Update static sign recognition model"
git -C $RepoPath push origin main
```

Continuous sign update (only after continuous candidate was promoted and checked):

```
Set-Location $TrainingRoot
.\sync_model_to_github.ps1 -RepoPath $RepoPath -ModelType Continuous
git -C $RepoPath status --short
git -C $RepoPath add SAHAYAK-Vercel-Deployment/backend/model/sahayak_sequence.joblib
git -C $RepoPath diff --cached --name-only
git -C $RepoPath commit -m "Update continuous sign recognition model"
git -C $RepoPath push origin main
```

Before committing, inspect git -C $RepoPath diff --cached --name-only and verify the staged list contains only the intended model path(s). If anything unexpected is staged, stop and unstage everything before adding only the correct paths.

Never push data/landmarks_2hand.csv, continuous_gesture_db.db, Code\gesture_db.db, .venv, model/backups, candidate models, or secret files. The remote backup branch preserves the previous GitHub tree before the new model commit.

7. Verify the live system after deployment

```
# After Render finishes deploying the pushed commit:
$status = Invoke-RestMethod "https://sahayak-backend-krh1.onrender.com/api/status"
$status.model_ready
$status.single_sign_labels
$status.sequence_model_ready
$status.sequence_sign_labels
```

- Confirm the appropriate model-ready value is True and that the new label appears in the correct label list without removing previous labels.
- Use the studio page to test the newly added sign and a selection of older static and continuous signs. Verify uncertainty handling and that the recognition/annotation UI remains responsive.
- Model-only changes normally trigger the backend deployment through GitHub. Frontend changes require their own reviewed source commit and Vercel deployment verification.

8. Rollback and troubleshooting

- Training validation fails: keep the active model unchanged, inspect label counts and the reported error, then collect more varied samples and retrain the candidate.
- A previous label is missing: do not promote; verify the full dataset/database is retained and rebuild the candidate.
- Local API does not load a model: inspect the traceback and n_features_in_ compatibility before deployment.
- Production does not show the new label: verify the pushed commit, wait for Render to finish, recheck /api/status, and inspect service logs. Do not make repeated speculative pushes.
- Model regression: restore the previous model from the local timestamped backup, sync only that active model, and push a normal rollback commit. Do not force-push main.

9. Guide map

Static guide: SAHAYAK_Static_Sign_Collection_Training_Deployment_Guide.pdf. **Continuous guide:** SAHAYAK_Continuous_Sign_Collection_Training_Deployment_Guide.pdf. **Post-training release checklist:** SAHAYAK_After_Training_Testing_New_Sign_Deployment_Guide.pdf. This master guide links the same safe workflow together. Presentation scripts and the science exhibition guide are separate content documents and do not need changes for the model-promotion procedure.